Doc. no. P1789R0
Date: 2019-06-17
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Library Support for Expansion Statements

Table of Contents

1. Revision History
    - Revision 0
2. 1 Introduction
3. 2 Stating the problem
    - Implementing `std::tuple::swap`
4. 3 Propose Solution
5. 4 Sample Implementation
6. 5 Formal Wording
7. 6 Acknowledgements
8. 7 References

Revision History

Revision 0

Original version of the paper for the 2019 pre-Cologne mailing.

1 Introduction

Expansion statements add compile-time iteration to C++20, but there are no compile-time sequences of integer that support this feature, for inuitive `for` loop indexing from the runtime world. There is an easy fix with a simple enhancement the library template `integer_sequence` to support the structured binding API.

2 Stating the problem

Expansion statements are a new language feature for C++20, approved by EWG in Kona, and due to land in Cologne, 2019. See P1306R1 Expansion statement for details. They allow for iteration over parameter packs. However, my very first attempt at an example of using an expansion statement came up short, due to the omission of a tiny piece of supporting library infrastructure.

1. Implementing `std::tuple::swap`

It is realtively straightforward to implement `tuple::swap` using a fold expression over the comma operator, but also somewhat of a hack. This is the kind of code we would like to be able to write more cleanly using an expansion statement.

```
template <class... TYPES>
constexpr
void tuple<TYPES...>::swap(tuple& other)
```

```
      noexcept((is_nothrow_swappable_v<TYPES> and ...))
{
   auto impl = [&, this]<size_t...INDEX>(index_sequence<INDEX...>) {
      ((void)swap(get<INDEX>(*this), get<INDEX>(other)), ...);
   };

   impl(index_sequence_for<TYPES...>{});
}
```

Note the intenral use of a lambda expression, purely to get at the parameter pack to fold. Also note that we must cast the call to `swap` to `void` in case users provide an ADL-discoverable `swap` function that returns a user defined type, that in turn provides an overload for the comma operator. Finally, I took the liberty of not replicating the exception specification on the lambda expression, but is that relying too heavily on compilers to optimize away the (unneeded) catch-and-abort logic?

We can eliminate the fold expression and worrying about the crazy corner cases in ADL-`swap` like so:

```
template <class... TYPES>
constexpr
void tuple<TYPES...>::swap(tuple& other)
   noexcept((is_nothrow_swappable_v<TYPES> and ...))
{
   auto impl = [&, this]<size_t...INDEX>(index_sequence<INDEX...>) {
      for...(size_t N : INDEX...) { swap(get<N>(*this), get<N>(other)); }
   };

   impl(index_sequence_for<TYPES...>{});
}
```

However, there is no easy way to eliminate the lamba expression, as we cannot iterate over an `integer_sequence` using just the facilities provided in P1306R1.

3 Propose Solution

We propose that the simplest way to resolve the concerns is to add the missing pieces that would enable use of `integer_sequence` in a structured binding. That is sufficient to support use in expansion statements, and is general enough to be a feature in its own right. With such support, the `tuple::swap` example simplifies to:

```
template <class... TYPES>
constexpr
void tuple<TYPES...>::swap(tuple& other)
   noexcept((is_nothrow_swappable_v<TYPES> and ...))
{
   for...(size_t N : index_sequence_for<TYPES...>{}) {
      swap(get<N>(*this), get<N>(other));
   }
}
```

Make `integer_sequence` Support Structured Bindings

`integer_sequence` is missing three things in order to support use in structured bindings:

- A partial specialization for `tuple_size`
- A partial specialization for `tuple_elememt`
- Overloads of `get<INDEX>()`

The first two bullets are fairly straightforward to implement. For the `get` function, we propose a single overload taking an `integer_sequence` by value, as it is an immutable empty type, and likewise returning its result by value.

4 Simple Implementation

```
template<class T, T... VALUES>
struct tuple_size<integer_sequence<T, VALUES...>>
     : integral_constant<size_t, sizeof...(VALUES)>
{ };

template<size_t I, class T, T... VALUES>
   requires I < sizeof...(VALUES)
struct tuple_element<I, integer_sequence<T, VALUES...>> {
  using type = T;
};


template<size_t I, class T, T... VALUES>
constexpr T get(integer_sequence<T, VALUES...>) noexcept {
   constexpr T index[]{VALUES...};
   return index[I];
}
```

5 Formal Wording

Make the following changes to the specified working paper:

20.2.1 Header `<utility>` synopsis [utility.syn]

1. The header contains some basic function and class templates that are used throughout the rest of the library.

```
#include <initializer_list> // see 17.10.1

namespace std {
  // 20.2.2, swap
  ...

  // 20.3, Compile-time integer sequences
  template<class T, T...>
    struct integer_sequence;
  template<size_t... I>
    using index_sequence = integer_sequence<size_t, I...>;

  template<class T, T N>
    using make_integer_sequence = integer_sequence<T, see below>;

  template<size_t N>
    using make_index_sequence = make_integer_sequence<size_t, N>;

  template<class... T>
    using index_sequence_for = make_index_sequence<sizeof...(T)>;

  // forward declaration for structured binding support
  template<class T> struct tuple_size;
  template<size_t I, class T> struct tuple_element;

  // structured binding support for integer_sequence
  template<class T, T...> struct tuple_size<integer_sequence<T, T...>>;
  template<size_t I, class T, T...> struct tuple_element<I, integer_sequence<T, T...>>;

  template<size_t I, class T, T...>
  constexpr T get(integer_sequence<T, T...>) noexcept;

  // 20.4, class template pair
  template<class T1, class T2>
```

```
    struct pair;

  ...

  // 20.4.4, tuple-like access to pair
  template<class T> struct tuple_size;
  template<size_t I, class T> struct tuple_element;

  template<class T1, class T2> struct tuple_size<pair<T1, T2>>;
  template<size_t I, class T1, class T2> struct tuple_element<I, pair<T1, T2>>;

}
```

20.3.4 Structured Binding Support [intseq.binding]

```
template<class T, T...>
  struct tuple_size<integer_sequence<T, T...>>: integral_constant<size_t, N> { };

template<size_t I, class T, T...>
  struct tuple_element<I, integer_sequence<T, T...>>::type
```

1. *Mandates:* `I < N` is `true`.
2. *Value:* The type `T`.

```
template<size_t I, class T, T...>
constexpr T get(integer_sequence<T, T...>) noexcept;
```

3. *Mandates:* `I < N` is `true`.
4. *Returns:* The `I`th member of the parameter pack `T...`.

6 Acknowledgements

Thanks to Vittorio Romeo and Daveed Vandevoorde for their insights into the contents of this paper.

7 References

- P1306R1 Expansion statements, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde